

ULTIMATE SDK

PROGRAMMER'S REFERENCE GUIDE

For the 1541 Ultimate-II, Ultimate 64, Ultimate 64 Elite
and the Commodore 64 Ultimate

First edition.

This guide documents the Ultimate SDK, a software development kit for the Commodore 64 and the Ultimate family of cartridges and machines.

Every figure, timing and behaviour in this book was measured against real hardware running firmware 3.15 unless it says otherwise. Where the firmware does something other than its documentation describes, this guide follows the firmware and says so.

Released under the MIT licence. Commodore, the Commodore logo and Commodore 64 are trademarks of their respective owners; this guide is not affiliated with or endorsed by them.

Contents

Introduction	v
1 GETTING STARTED	2
0.1 What You Need	2
0.2 Switching the Command Interface On	2
0.3 Your First Program In BASIC	2
0.4 Your First Program In C	3
0.5 Your First Program In Assembly	4
0.6 When Nothing Happens	5
2 PROGRAMMING IN BASIC	8
0.7 Installing The Wedge	8
0.8 The Twenty-Three Keywords	8
0.9 Files And Directories	9
0.10 The RAM Expansion	10
0.11 Turbo	11
0.12 Talking To The Ultimate Directly	12
0.13 What BASIC Cannot Do	12
3 PROGRAMMING IN C	15
0.14 Building Against The SDK	15
0.15 The Shape Of Every Call	15
0.16 Buffers Belong To You	16
0.17 Result Codes	16
0.18 Capability Detection	16
0.19 A Worked Example	17
4 PROGRAMMING IN ASSEMBLY	19
0.20 What Assembly Callers Get	19
0.21 The Calling Convention	19
0.22 The Shared Variable Block	20
0.23 Zero Page, And Moving It	20
0.24 Which Calls Are Bounded In Time	20
5 OTHER TOOLCHAINS	23
0.25 Why A Jump Table	23
0.26 Loading The Blob	23

0.27 Calling Through The Table	23
0.28 The Parameter Block	23
0.29 Driving It From BASIC With POKE And SYS	24
6 THE SERVICES	26
0.30 Files And Directories	26
0.31 Loading And Saving	26
0.32 The RAM Expansion	26
0.33 Processor Speed	27
0.34 The Screen Palette	27
0.35 Network Sockets	27
0.36 Fetching Over HTTP	28
7 APPENDICES	30
.1 Appendix A: Every BASIC Keyword	30
.2 Appendix B: Result Codes	31
.3 Appendix C: Where To Find The Rest	31

INTRODUCTION

Welcome. If you have an Ultimate cartridge or an Ultimate 64, you own a Commodore 64 that can reach a great deal more than a Commodore 64 ever could — files on a USB stick, sixteen megabytes of RAM expansion, a processor that runs up to forty-eight times faster than the one Commodore shipped, and a network connection to the rest of the world. This guide is about writing programs that use all of it.

The **ULTIMATE SDK** is a library, and it comes in the language you already write in. If you write BASIC, it arrives as new keywords you type at the `READY.` prompt. If you write C with cc65, it is a header and a library to link. If you write assembly, it is a set of entry points and a documented calling convention. And if you use a toolchain that cannot link any of those, there is a standalone binary with a jump table that any assembler can call.

The four routes are not four implementations. They are one implementation, reached four ways, and every one of them is tested against the same hardware.

What is in it

The SDK started as a wrapper around the Ultimate Command Interface — the hardware mailbox at `$DF1C` that a program uses to ask the cartridge to do something — and it has grown past that. Three of the things it does are not command-interface operations at all, because the machine offers them a different way:

- **Files and directories**, over the command interface.
- **Loading and saving**, taking the firmware's fast path when there is one.
- **The RAM expansion**, both by direct memory access — which is not a command at all, it is the REU's own registers — and file to expansion without the bytes passing through the C64.
- **Processor speed** on an Ultimate 64, which is memory-mapped I/O and has no command behind it either.
- **The screen palette**, all sixteen colours, live.
- **Network sockets and HTTP**, so a C64 can fetch a file from a web server.

How to read this

Chapter 1 gets you running something in the next ten minutes, whichever language you write in. Chapters 2 to 5 take one language each — BASIC, C, assembly, and everything else — and are meant to be read start to finish once and then dipped into. The chapters after that take one service each and can be read in any order, or not at all until you need them.

Everything is small enough to type in. The programs in this book are short on purpose: a listing you can enter in a minute and watch work teaches more than one you copy from a disk and take on trust.

Note: This guide describes behaviour that was measured, not behaviour that was assumed. Where a figure appears — how many cycles something costs, how long a connection takes to fail — it came off a real machine, and the guide says which one.

A word about the name

The library is called the Ultimate SDK rather than the Ultimate 64 SDK because most of it works on every Ultimate from the 1541 Ultimate-II upward, and not only on the Ultimate 64. Where something needs a particular machine — the turbo registers need an Ultimate 64 — the guide says so at the point it matters, and the SDK itself tells you at run time rather than guessing.

CHAPTER 1

GETTING STARTED

- What you need
- Switching the command interface on
- Your first program in BASIC
- Your first program in C
- Your first program in assembly
- When nothing happens

WHAT YOU NEED

Three things, and you may have all of them already.

An Ultimate. Any of them: a 1541 Ultimate-II or II+ plugged into a Commodore 64, an Ultimate 64 or 64 Elite, or a Commodore 64 Ultimate. The SDK speaks to all of them through the same five addresses and asks each one what it can do rather than assuming.

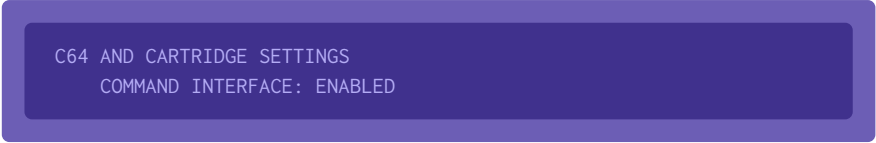
Firmware 3.15 or later, which is where the command interface settled into the shape this guide describes. Older firmware works for most of it; the guide flags the places where it does not.

The command interface switched on. This is the one that catches everybody, and it is the next section.

SWITCHING THE COMMAND INTERFACE ON

The Ultimate ships with its command interface *disabled*. A program that tries to use it on a machine where nobody has turned it on will find nothing there — correctly, and quickly, but it will find nothing.

Press **F5** to bring up the Ultimate menu, then find:



```
C64 AND CARTRIDGE SETTINGS
COMMAND INTERFACE: ENABLED
```

Set it to ENABLED. That is the whole of the setup.

Note: You do not have to do this for the RAM expansion or for turbo. Those two reach the hardware directly and work whether or not the command interface is on — though they have settings of their own in the same menu.

YOUR FIRST PROGRAM IN BASIC

The BASIC route needs no compiler and no cross-development machine. Load the wedge, run it, and BASIC has twenty-three new keywords.

```
LOAD"UCI",8  
RUN
```

The wedge installs itself and hands you back the `READY.` prompt. Now type this, exactly as shown, and press **RETURN**:

```
PRINT UREU
```

If you have a RAM expansion switched on, the machine answers with its size in 64K banks — 32 for a two-megabyte expansion. If you have not, it answers 0. Either way, your Commodore 64 has just asked the cartridge a question and been told the truth.

Now something with a little more in it. Type:

```
UDIR
```

and the directory of whatever the Ultimate has mounted scrolls up the screen. No disk drive was involved.

A short program

Type `NEW`, then this. Remember **RETURN** at the end of every line.

```
10 IF UREU = 0 THEN PRINT "NO EXPANSION": END  
20 PRINT "EXPANSION IS"; UREU * 64; "K"  
30 PRINT "TURBO INDEX IS"; UTURBO
```

Type `RUN`. Three lines, and the program has interrogated two pieces of hardware that a stock Commodore 64 does not have.

YOUR FIRST PROGRAM IN C

You will need `cc65`, and you will build on a modern machine and copy the result across. From the `SDK` directory:

```
make lib
```

That produces `bindings/cc65/build/ultimate.lib`. Now a program:

```
#include <stdio.h>
#include <ultimate.h>

int main(void)
{
    uint16_t banks;

    if (ultimate_init() != ULTIMATE_OK) {
        printf("no ultimate here
n");
        return 1;
    }

    banks = ultimate_reu_size();
    printf("expansion: %u banks
n", banks);
    return 0;
}
```

Build it, pointing the compiler at the SDK's headers and library:

```
cl65 -t c64 -I include -o hello.prg hello.c \
    bindings/cc65/build/ultimate.lib
```

Copy `hello.prg` to the Ultimate and run it.

Note: `ultimate_init()` is worth calling first and worth checking. It verifies the interface is really there, clears anything a previous program left behind, and puts the transport into a known state. Every other call assumes it has run.

YOUR FIRST PROGRAM IN ASSEMBLY

Assembly callers need no C runtime at all — no software stack to set up, no start-up code to run. Include the SDK's declarations and call the entry points.

```

.include "ultimate.inc"

jsr ultimate_init
cmp #ULTIMATE_OK
bne no_ultimate

jsr ultimate_reu_size ; A = low, X = high, in 64K banks
sta banks
stx banks + 1
rts

```

Assemble and link it against the same library the C route uses. The whole assembly contract — which registers carry what, which variables an entry point reads, and which of them are bounded in time — is Chapter 4.

WHEN NOTHING HAPPENS

Four things account for almost every "it did not work", in the order they are worth checking.

The command interface is off

The commonest by far. `ultimate_init()` returns `ULTIMATE_ERR_NO_DEVICE` and every later call fails the same way. Go and turn it on: see the second section of this chapter.

Something crashed mid-command

A program that stopped in the middle of talking to the Ultimate can leave the interface holding a half-finished exchange, and the next program to run finds it busy. This is what `uci_abort()` is for, and `ultimate_init()` calls it for you — so simply running your program again usually clears it.

From BASIC, a bare UCI with no arguments does the same thing:

```
UCI
```

You asked for something this machine has not got

Turbo needs an Ultimate 64 and the `TURBO CONTROL` setting; the RAM expansion needs to be enabled and given a size. The SDK will tell you rather than pretend: `ultimate_turbo_available()` answers plainly, and `ultimate_reu_size()` returns zero when there is nothing there.

The reply did not fit

If a call comes back `ULTIMATE_ERR_TRUNCATED`, the command worked and your buffer was too small. The count that comes back tells you how much was kept.

Note: Every entry point in the SDK returns a result code, and it is worth checking every one. The codes are a small fixed set — Appendix B lists them all — and `ultimate_strerror()` turns any of them into text you can print.

CHAPTER 2

PROGRAMMING IN BASIC

- Installing the wedge
- The twenty-three keywords
- Files and directories
- The RAM expansion
- Turbo
- Talking to the Ultimate directly
- What BASIC cannot do

INSTALLING THE WEDGE

The wedge comes two ways, and they are the same program.

As a program. LOAD"UCI", 8 then RUN. It installs itself into the 4K block at \$C000, which BASIC does not use, and leaves your 38911 bytes free exactly as they were.

As a cartridge. Mount uci.crt from the Ultimate's menu and the wedge is there the moment the machine starts, with nothing to load. The cartridge costs BASIC its 8K: you get 30719 bytes free instead of 38911.

Either way the machine greets you and the keywords are live. They survive NEW, they survive RUN/STOP + **RESTORE**, and they survive a program crashing. They do not survive a reset.

Note: The wedge takes over four of BASIC's own vectors — the ones for tokenising, listing, statement dispatch and expression evaluation. Every other wedge and fast-loader that does the same thing will fight with it. If you use one, load the SDK's wedge last.

THE TWENTY-THREE KEYWORDS

They fall into four groups, and you can ignore three of them until you need them.

Asking how things went

Nothing in this wedge stops your program with an error. A command that fails sets a result code and returns, because a demo dropping to READY. in the middle of a part is worse than one that quietly did nothing. So every command is followed, when it matters, by a look at UERR.

UERR	the result of the last command. 0 is success
UDEV	the raw code the Ultimate reported, for diagnosis
ULEN	how many bytes came back
UST\$	the status text, exactly as the Ultimate sent it
UDAT\$	the reply as a string, up to 255 bytes
UBYTE(N)	byte N of the reply, or 0 past the end

Files, and the RAM expansion

UDIR	print the current directory
ULOAD <i>name\$</i> [, <i>addr</i>]	load a PRG; no address means the file's own
UBLOAD <i>name\$, addr, len</i>	load raw bytes, and not one past <i>len</i>
USAVE <i>name\$, addr, len</i>	memory to a file
USTASH <i>addr, reu, len</i>	C64 memory into the expansion
UFETCH <i>addr, reu, len</i>	and back out of it
UREU	the expansion's size in 64K banks, 0 for none

Speed

UTURBO <i>n</i>	set the processor speed index
UTURBO	as a function, the current index — or 255 when the machine's owner has turned turbo off

Everything else

UCI <i>t, c</i> [, <i>args</i>]	send any command to any target
UCI	on its own, rescue a wedged interface
UW(<i>x</i>)	force a 16-bit argument
UL(<i>x</i>)	force a 32-bit argument
UDOS1 UDOS2 UNET UCTRL UIEC UHTTP	the six target numbers, by name

FILES AND DIRECTORIES

UDIR prints the directory and is the quickest way to see that everything is working:



```
UDIR
```

Loading is the interesting one, because the Ultimate has a fast path and the SDK takes it when it can. This loads a file to the address stored in the file itself, exactly as `LOAD"X", 8, 1` would:

```
10 ULOAD "GAME"  
20 IF UERR THEN PRINT "COULD NOT LOAD": END
```

Give it an address and it loads there instead:

```
10 ULOAD "SPRITES",832  
20 IF UERR THEN PRINT "NO SPRITES": END
```

UBLOAD is the one to use for data rather than programs. It interprets nothing, strips no header, and stops at the length you give it:

```
10 UBLOAD "MUSIC.BIN",49152,4096
```

Note: Filenames go to the Ultimate exactly as you type them. Uppercase PETSCII and uppercase ASCII are the same bytes, so a name typed at a C64 arrives as the Ultimate expects it, and the filesystem does not care about case anyway.

THE RAM EXPANSION

If there is an expansion fitted, you have up to sixteen megabytes to put things in — and moving bytes in and out of it costs almost nothing, because the REU does it with the processor halted rather than a byte at a time.

Always ask first. UREU returns the size in 64K banks and zero when there is nothing there, so one test covers both questions:

```
10 B = UREU  
20 IF B = 0 THEN PRINT "NO EXPANSION": END  
30 PRINT "EXPANSION IS"; B * 64; "K"
```

Now put something in it and get it back. This copies the top of the screen into the

expansion, clears the screen, and puts it back:

```
10 IF UREU = 0 THEN PRINT "NEED AN REU": END
20 USTASH 1024,0,1000
30 PRINT CHR$(147)
40 FOR I = 1 TO 2000: NEXT
50 UFETCH 1024,0,1000
```

Line 20 sends 1000 bytes from screen memory at 1024 to expansion address 0. Line 50 brings them back. Both are instant.

Note: The expansion address is a full 24-bit number, so it can reach all sixteen megabytes. BASIC handles that fine — `USTASH 1024, 1048576, 256` is a perfectly good line.

Careful: Writing past the end of the expansion does not fail — it wraps round to the beginning and quietly overwrites what is there. That is exactly how `UREU` measures the size, and it is why asking first is worth the one line.

TURBO

On an Ultimate 64 the processor can run faster than 1MHz, and `UTURBO` sets it. The catch is that this is the machine owner's decision, not your program's: unless `TURBO CONTROL` is set in the Ultimate's menu, the registers are not there at all.

```
10 UTURBO 3
20 IF UERR THEN PRINT "NO TURBO HERE"
30 PRINT "SPEED INDEX IS"; UTURBO
```

Index 0 is 1MHz and index 3 is 4MHz on every machine that has turbo. Above 3 the numbers mean different speeds on different machines, so a program that ships to other people should stay at or below 3 or ask.

The function form answers 255 when turbo is unavailable. That is not a speed — the index is only four bits — so you can test for it.

TALKING TO THE ULTIMATE DIRECTLY

The wedge wraps the operations most programs want. Everything else — and there are around a hundred commands — is reachable with UCI, which sends any command to any target.

```
10 UCI UCTRL, 1
20 PRINT UDAT$
```

That sends command 1 to the control target, which identifies itself, and prints what came back.

Arguments follow the command, and the wedge sends each number as a single byte because that is what nearly every command wants. When a command wants something wider, say so with `UW()` for sixteen bits or `UL()` for thirty-two:

```
10 UCI UDOS1, 16, UW(8192)
```

Note: The SDK knows the shape of many commands and will widen arguments for you without being told. `UW()` and `UL()` are the escape hatch for a command newer than the SDK's table, and they always win over it.

WHAT BASIC CANNOT DO

Two limits worth knowing before they surprise you.

IF X THEN UCI does not work

BASIC's IF jumps straight into the ROM's own statement dispatcher, which rejects any token above NEW. Every extension that adds statement keywords to BASIC V2 has this hole. So this does not work:

```
10 IF X = 1 THEN ULOAD "GAME"
```

and this does:

```
10 IF X = 1 THEN 100
. . .
100 ULOAD "GAME"
```

Functions are unaffected, because expression evaluation does come back through the wedge. IF UERR THEN . . . is fine, and so is IF UREU = 0 THEN . . .

Strings are 255 bytes

UDAT\$ gives you the reply as a string, clipped at 255 bytes because that is the longest string BASIC has. When a reply is longer than that, use UBYTE(N) to walk it, or ULOAD to put it somewhere you can PEEK.

CHAPTER 3

PROGRAMMING IN C

- Building against the SDK
- The shape of every call
- Buffers belong to you
- Result codes
- Capability detection
- A worked example

BUILDING AGAINST THE SDK

Build the library once:

```
make lib
```

then compile against include/ and link ultimate.lib:

```
cl65 -t c64 -Osir -I include -o prog.prg prog.c \  
    bindings/cc65/build/ultimate.lib
```

One header covers the whole library:

```
#include <ultimate.h>
```

If you need the transport itself — to send a command the SDK does not wrap — add `<uci.h>` as well. Most programs never do.

THE SHAPE OF EVERY CALL

Almost every function in the SDK looks like this:

```
uint8_t ultimate_something(. . .);
```

It returns a result code, and `ULTIMATE_OK` is zero. Anything it produces comes back through a pointer you passed in. There are no global error variables to check, nothing is returned in a static buffer, and no call allocates memory.

```
uint16_t got;  
uint8_t err;  
  
err = ultimate_read(buf, sizeof buf, &got);  
if (err != ULTIMATE_OK)  
    return err;
```

BUFFERS BELONG TO YOU

The SDK has no heap and no hidden buffers. Every byte it produces lands in memory you own, and it never keeps a pointer after the call returns. That is what makes it usable from an interrupt-driven demo, and it is why every call that produces data takes both a buffer and its size.

Careful: The size you pass is the size of the buffer, not the number of bytes you want. If a reply is larger, you get `ULTIMATE_ERR_TRUNCATED`, the count tells you how much was kept, and nothing is written past the end.

RESULT CODES

There are eleven, they never change, and Appendix B lists them. Three are worth knowing straight away:

<code>ULTIMATE_OK</code>	it worked
<code>ULTIMATE_END</code>	there is no more — the end of a directory, or a socket the far end has closed. Not a failure
<code>ULTIMATE_ERR_TRUNCATED</code>	it worked, and your buffer was too small

Turn any of them into text with:

```
printf("%s\n", ultimate_strerror(err));
```

CAPABILITY DETECTION

Different machines have different targets, and firmware versions differ too. Ask once, at start-up, rather than assuming:

```
ultimate_capabilities caps;

ultimate_detect(&caps);
if (ultimate_has_http(&caps)) { /* fetch things */ }
if (ultimate_has_network(&caps)) { /* sockets */ }
```

The hardware that is not reached through the command interface answers separately, because it is not a target: `ultimate_turbo_available()` and `ultimate_reu_size()` tell you about the processor speed registers and the RAM expansion.

A WORKED EXAMPLE

This prints the directory, one entry at a time. It is the pattern for every walk in the SDK: start it, take entries until `ULTIMATE_END`, and check the result of the call that ended it.

```
#include <stdio.h>
#include <ultimate.h>

int main(void)
{
    char name[64];
    uint8_t attrib;
    uint8_t err;

    if (ultimate_init() != ULTIMATE_OK) {
        printf("no ultimate\n");
        return 1;
    }

    err = ultimate_opendir();
    while (err == ULTIMATE_OK) {
        err = ultimate_readdir(name, sizeof name, &attrib);
        if (err != ULTIMATE_OK)
            break;
        printf("%s%s\n", name,
              (attrib & DOS_ATTR_DIR) ? "/" : "");
    }

    if (err != ULTIMATE_END)
        printf("stopped: %s\n", ultimate_strerror(err));
    return 0;
}
```

Note: A directory walk is one live exchange with the Ultimate. Do not issue any other command in the middle of one — the block boundaries between entries are the only thing separating one name from the next. If you need to stop early, call `uci_abort()`.

CHAPTER 4

PROGRAMMING IN ASSEMBLY

- What assembly callers get
- The calling convention
- The shared variable block
- Zero page, and moving it
- Which calls are bounded in time

WHAT ASSEMBLY CALLERS GET

The SDK is written in 6502 assembly, so an assembly caller is talking to it directly rather than through a translation layer. There is no C runtime to start, no software stack to set up, and nothing to initialise before the first `jsr`. A cartridge that starts at \$8000 can call `ultimate_init` as its first instruction.

```
.include "ultimate.inc"
```

That one file declares every entry point and every shared variable.

THE CALLING CONVENTION

Three rules cover nearly everything:

- A result comes back in A, and `ULTIMATE_OK` is zero.
- A single pointer argument goes in A/X, low byte first.
- Anything wider than that goes in the shared variable block.

So the simple calls look like this:

```
lda #<name
ldx #>name
jsr ultimate_chdir
cmp #ULTIMATE_OK
bne failed
```

and the ones that need more set the variables first:

```
lda #<buffer
sta ult_buf
lda #>buffer
sta ult_buf + 1
lda #<64
sta ult_buflen
lda #>64
sta ult_buflen + 1
jsr ultimate_getpath
```

Appendix C lists every entry point with what it reads and what it returns.

THE SHARED VARIABLE BLOCK

The SDK needs 143 bytes of RAM, and by default the linker places them in BSS like any other variables. A cartridge cannot do that — its code is in ROM and the RAM it may touch is the programmer's decision — so the whole block can be placed instead:

```
ca65 -D UCI_VARS=$CF00 . . .
```

Every variable then becomes an equate at a fixed offset from that address, and there is no address in the SDK a cartridge author cannot move.

ZERO PAGE, AND MOVING IT

Four bytes, and you choose where. The default is \$FB-\$FE, which is free on a stock C64 — the KERNAL uses it only for tape — and which cc65's runtime does not claim.

```
ca65 -D UCI_ZP=$A3 . . .
```

Note: Four bytes is the whole of it. The SDK installs no interrupt handler, requires no interrupts to be enabled, and never writes anywhere you did not tell it to.

WHICH CALLS ARE BOUNDED IN TIME

Every entry point completes or returns `ULTIMATE_ERR_TIMEOUT` — with three deliberate exceptions, and they are exceptions because the hardware leaves no choice.

`ultimate_net_connect`, `ultimate_net_udp` and `ultimate_http_exchange` each contain a name lookup and a TCP connection. A connection to an address with nothing at it was measured at 30.8 seconds before the firmware gave up, and the SDK's timeout budget is a single byte counting polls — it tops out near one second. No value of it can express thirty. So those three wait, relying on the firmware's own limit, and put your timeout budget back when they return.

```
jsr uci_get_timeout_a ; A = the current budget
pha
lda #UCI_TIMEOUT_FOREVER
jsr uci_set_timeout_a
```

```
; ... something slow ...  
pla  
jsr uci_set_timeout_a
```

That is the pattern the SDK uses itself, and it is the one to copy if you drive a slow command through the generic form.

CHAPTER 5

OTHER TOOLCHAINS

- Why a jump table
- Loading the blob
- Calling through the table
- The parameter block
- Driving it from BASIC with POKE and SYS

WHY A JUMP TABLE

KickAssembler, ACME, 64tass, Oscar64, llvm-mos and KickC cannot link a ca65 object file. Rather than port the SDK to each of them — six implementations to keep in step, and six chances to disagree — there is one binary with a jump table at the front. Every toolchain can jsr an address.

It is not a port. It is the same object files the C library is built from, linked differently.

LOADING THE BLOB

Load `ultimate.bin` at `$8000` and check the signature before calling anything:

```
lda $C000
cmp #$D5    ; 'U' in PETSCII
bne no_sdk
```

The binary is relocatable: a table of the addresses that need adjusting ships with it, so you can move it somewhere else and fix it up.

CALLING THROUGH THE TABLE

The table starts four bytes in, three bytes per entry, in a published order that is append-only — new entries go on the end and existing ones never move.

```
jsr $8004    ; +$04 uci_init
jsr $801C    ; +$1C ultimate_init
```

Appendix C has every offset.

THE PARAMETER BLOCK

A caller cannot pass a filename in `A/X`, so the entries that need more take their arguments from a page-aligned block that follows the table. You write the fields, call the entry, and read the result out of the same block.

+\$00	the result code
+\$01	the raw device code
+\$03	an address argument
+\$05	a length, in and out
+\$29	the filename
+\$51	the reply buffer

DRIVING IT FROM BASIC WITH POKE AND SYS

The block is page-aligned for a reason: it is the only calling convention BASIC can express without a wedge. POKE the fields, SYS the entry, and PEEK the answer.

```
10 B = 32768 : P = B + 256
20 SYS B + 28 : REM ULTIMATE_INIT
30 SYS B + 175 : REM REU_SIZE
40 PRINT "BANKS:"; PEEK(P+5) + PEEK(P+6)*256
```

Note: This is the long way round if you have the wedge — UREU does the same thing in four characters. It is here because a program that ships the blob for its assembly code can drive it from its BASIC loader too, without shipping the wedge as well.

CHAPTER 6

THE SERVICES

- Files and directories
- Loading and saving
- The RAM expansion
- Processor speed
- The screen palette
- Network sockets
- Fetching over HTTP

FILES AND DIRECTORIES

The Ultimate's own filesystem, reached by name. Open a file, read or write it, seek in it, close it — the operations you would expect, with two things worth knowing.

A directory walk is **one live exchange**: `ultimate_opendir()` then `ultimate_readdir()` until it answers `ULTIMATE_END`, with no other command in between. And a filename goes on the wire exactly as you gave it: the SDK converts nothing, because uppercase PETSCII and uppercase ASCII are the same bytes and the filesystem does not care about case.

LOADING AND SAVING

`ultimate_load()` takes the firmware's fast path when the machine has one — the bytes go straight into C64 memory without passing through the command interface at all — and falls back to the ordinary route when it does not. You do not choose; the SDK does, and tells you where the file ended.

`ultimate_bload()` is the one for data: no header consumed, nothing interpreted, and not one byte past the limit you set.

THE RAM EXPANSION

Two different things share one name here, and the difference matters.

C64 memory to the expansion and back is the REU's own direct memory access. There is no command involved: the SDK programs eight registers and the transfer happens with the processor halted, so by the next instruction it is already done. Nothing to poll, nothing to wait for.

A file to the expansion is a command, and it is the one that cannot be done any other way — the bytes never pass through the C64 at all. Sixteen megabytes of assets can be loaded without staging them through 64K.

`ultimate_reu_size()` answers how much there is, in 64K banks, and zero when there is none. It measures rather than asks, because nothing in the protocol reports it: the expansion aliases, so the first power-of-two boundary whose write comes back round to the start is the size.

Note: 16MB is the ceiling whatever the machine has, because the expansion's address registers are 24 bits. A machine with more RAM than that cannot reach the rest through this interface.

PROCESSOR SPEED

Memory-mapped, not a command, and only on an Ultimate 64. The index runs from 0 to 15; 0 is 1MHz and 3 is 4MHz on every machine that has it. Above 3 the same index means different speeds on different models, so the SDK passes the number through and does not pretend to know megahertz.

Whether turbo exists at all is the machine owner's decision. Both registers read \$FF when it has been switched off, which makes availability testable rather than assumed.

There is a second half to it: bit 7 turns off badlines, which stops the video chip stealing cycles from the processor. On a fixed loop that is worth about 6.3% — measured, and it agrees with the arithmetic.

THE SCREEN PALETTE

All sixteen colours, live, on firmware newer than 3.15. This affects the running palette only — not flash, not a file — so a program that crashes half way through a colour cycle cannot leave the machine looking wrong permanently.

A whole-palette rotation costs about a quarter of a frame, so colour cycling directly is comfortably affordable.

NETWORK SOCKETS

TCP and UDP, with three things that decide how you use them.

A read never waits. One issued straight after connecting reports "nothing yet" even when the far end has already replied. Callers poll — which is the right answer on a C64, because a blocking read would freeze the machine for as long as the other end stayed quiet.

The end of the stream is `ULTIMATE_END`, the same code a directory ends with, and it is reported exactly once. After it the handle is already gone, so do not close it.

Connecting can take thirty seconds. 48 to 75 milliseconds to a host that is up, and 30.8 seconds to an address with nothing at it before the firmware gives up.

```
for (;;) {
    err = ultimate_net_read(h, buf, sizeof buf, &got);
    if (err == ULTIMATE_END) break; /* far end hung up */
    if (err != ULTIMATE_OK) return err;
    if (got == 0) continue; /* nothing yet */
    /* ... use buf[0..got-1] ... */
}
```

FETCHING OVER HTTP

`ultimate_http_get()` is the whole of the common case: it builds the request, runs it, and gives the firmware's slot back whether it worked or not.

```
uint16_t got;
uint8_t err;

err = ultimate_http_get(url, buf, sizeof buf, &got);
```

ULTIMATE_OK means the server answered below 400. A 404 or a 500 is ULTIMATE_ERR_DEVICE with the number in `ultimate_device_code()` — the request itself worked, and the body of the error page is in your buffer.

Careful: A URL written as a plain C string literal arrives at the server uppercased. cc65 translates source characters into PETSCII, and an HTTP path is case sensitive. Build the URL as bytes, or fold it back at run time.

Note: A reply larger than your buffer is truncated and the rest is gone. Nothing in the protocol offers a range request or a second block, so size the buffer for what you expect.

CHAPTER 7

APPENDICES

- A — Every BASIC keyword
- B — Result codes
- C — Where to find the rest

APPENDIX A: EVERY BASIC KEYWORD

Token values are a file format — a tokenised program stores the number, not the name — so this table is append-only and these numbers never change.

Token	Keyword	
\$CC	UCI	any command, any target; alone, rescue the interface
\$CD	UERR	the last result code
\$CE	UDEV	the raw device code
\$CF	ULEN	reply length
\$D0	UST\$	status text
\$D1	UDAT\$	reply as a string
\$D2	UBYTE(	byte n of the reply
\$D3	UW(	force 16 bits
\$D4	UL(	force 32 bits
\$D5	UDOS1	target 1
\$D6	UDOS2	target 2
\$D7	UNET	target 3
\$D8	UCTRL	target 4
\$D9	UIEC	target 5
\$DA	UHTTP	target 6
\$DB	UTURBO	set or read the speed index
\$DC	ULOAD	load a PRG
\$DD	UBLOAD	load raw bytes
\$DE	USAVE	memory to a file
\$DF	UDIR	print the directory
\$E0	USTASH	C64 memory into the expansion
\$E1	UFETCH	and back out
\$E2	UREU	the expansion's size in banks

APPENDIX B: RESULT CODES

0	ULTIMATE_OK	success
1	ULTIMATE_ERR_NO_DEVICE	no command interface found
2	ULTIMATE_ERR_TIMEOUT	it did not answer in time
3	ULTIMATE_ERR_PROTOCOL	a transport-level violation
4	ULTIMATE_ERR_NOT_SUPPORTED	this firmware has not got it
5	ULTIMATE_ERR_INVALID_ARGUMENT	your mistake, caught early
6	ULTIMATE_ERR_IO	filesystem or network failure
7	ULTIMATE_ERR_DEVICE	the target reported a failure
8	ULTIMATE_ERR_TRUNCATED	it worked; the buffer was small
9	ULTIMATE_ERR_ABORTED	the command was abandoned
10	ULTIMATE_END	no more — and not a failure

`ultimate_strerror()` turns any of them into a short, stable description.

APPENDIX C: WHERE TO FIND THE REST

This guide is the friendly half. The reference half lives with the source, and is generated from the same definitions the code is:

<code>docs/uci.md</code>	the protocol itself, with every fact sourced and measured behaviour separated from documented behaviour
<code>docs/asm-abi.md</code>	the assembly contract, entry point by entry point
<code>docs/api-design.md</code>	why the API looks like this
<code>docs/compatibility.md</code>	hardware, firmware, speeds, known issues
<code>docs/generated/</code>	every constant, every keyword and every command the tests exercise — generated, so never out of step
<code>bindings/blob/README.md</code>	the jump table and parameter block, offset by offset